

UNIVERSITÀ
DEGLI STUDI
DI PADOVA



DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

CORSO DI PROGRAMMAZIONE DI SISTEMI EMBEDDED

Report ChefGPT

Fabbro Joao Leonardo

Riolmi Rossetto Daniele

Tonin Luca

Zavarise Mirco

ANNO ACCADEMICO 2024-2025

Indice

1	NNAPI	3
1.1	Panoramica dell'API Neural Networks	3
1.1.1	Introduzione e Scopo	3
1.2	Modello di Programmazione NNAPI	4
1.2.1	Astrazioni Principali	4
1.2.2	Modello: Operandi e Operazioni	5
1.3	Vantaggi e Svantaggi dell'Inferenza On-Device	6
2	TensorFlow Lite	7
2.1	Cos'è e a cosa serve	7
2.1.1	Workflow dello sviluppatore	7
2.2	Esecuzione dell'inferenza	8
2.2.1	Piattaforme supportate	9
2.3	Differenze principali da TensorFlow	12
3	LiteRT	15
3.1	Cos'è LiteRT	15
3.1.1	Funzionalità principali	15
3.1.2	Flusso di lavoro di sviluppo	16
3.1.3	Conversione dei modelli	16
3.1.4	Esecuzione dell'inferenza	18
3.1.5	Migrazione da TensorFlow Lite	19
3.2	Quantizzazione in LiteRT	19
3.2.1	Specifiche tecniche della quantizzazione a 8 bit	21
3.2.2	Operatori quantizzati	21
3.2.3	Quantization Debugger	21
3.3	Metadati LiteRT	22
3.4	Aggiunta di Metadati	23

3.5	Visualizzazione e Lettura dei Metadati	23
4	MediaPipe	25
4.1	MediaPipe Solutions	25
4.2	MediaPipe API	27
4.3	OutputHandler	28
4.4	ProgressListener<String>	28
4.5	Gestione risorse	29
5	ExecuTorch	31
5.1	Introduzione	31
5.2	Flusso Operativo	31
5.3	Esportazione e Trasformazione del Modello	32
5.4	Backend Hardware Supportati	32
5.5	Esecuzione su Dispositivo	33
5.6	Costruzione da Sorgente	33
5.7	Integrazione del Runtime e Ottimizzazioni	33
	Bibliografia	35

Capitolo 1

NNAPI

1.1 Panoramica dell'API Neural Networks

1.1.1 Introduzione e Scopo

L'API Android Neural Networks (NNAPI) è un'API C per Android progettata per eseguire operazioni di calcolo intensivo per il machine learning sui dispositivi Android. Il suo scopo è fornire un livello di base di funzionalità per framework di machine learning di livello superiore, come TensorFlow Lite e Caffe2, che creano e addestrano reti neurali. NNAPI consente operazioni di inferenza con accelerazione hardware sui dispositivi Android supportati. Generalmente, le app non utilizzano direttamente NNAPI, ma preferiscono utilizzare framework di livello superiore che, a loro volta, potrebbero utilizzare NNAPI per l'accelerazione hardware.

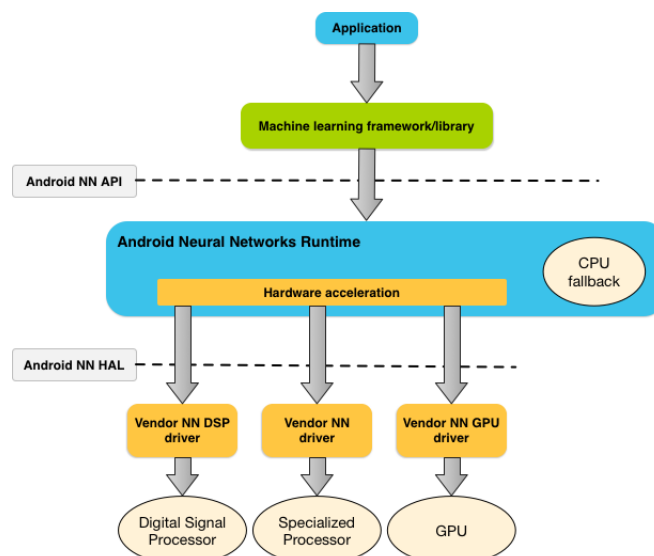


Figura 1.1: schema NNAPI [1]

NNAPI è stata introdotta in Android 8.1 (livello API 27). Tuttavia, è stata ritirata ("deprecated") in Android 15. La ragione del ritiro è che il campo del machine learning on-device (ODML) è progredito rapidamente, richiedendo strumenti e infrastrutture che si aggiornano frequentemente per supportare progressi come i modelli di transformer e di diffusione. Sebbene sia ancora possibile utilizzare NNAPI, le fonti indicano che la maggior parte dei dispositivi in futuro utilizzerà il backend della CPU e raccomandano la migrazione a soluzioni alternative per i workload critici in termini di prestazioni, come il runtime GPU di TF Lite.

Il modulo NNAPI Runtime è una libreria condivisa che si trova tra un'app e i driver di backend. Questo modulo è aggiornabile, il che significa che può ricevere aggiornamenti alle funzionalità al di fuori del normale ciclo di release di Android. In base ai requisiti di un'app e alle funzionalità hardware di un dispositivo Android, il runtime NNAPI può distribuire in modo efficiente il carico di lavoro di calcolo tra i processori on-device disponibili. Questi processori possono includere hardware dedicato per le reti neurali, unità di elaborazione grafica (GPU) e processori di segnale digitale (DSP). Per i dispositivi Android che non dispongono di un driver del fornitore specializzato, il runtime NNAPI esegue le richieste sulla CPU.

1.2 Modello di Programmazione NNAPI

1.2.1 Astrazioni Principali

Per eseguire calcoli utilizzando NNAPI, è necessario costruire un grafico diretto che definisce le operazioni da eseguire. Questo grafico, combinato con i dati di input (come pesi e bias), forma il modello per la valutazione. NNAPI utilizza quattro astrazioni principali:

- **Modello:** un grafico di calcolo delle operazioni matematiche e dei valori costanti appresi tramite addestramento. Queste operazioni sono specifiche per le reti neurali.
- **Compilazione:** rappresenta una configurazione per la compilazione di un modello in codice di livello inferiore.
- **Memoria:** rappresenta memoria condivisa e buffer utilizzati per memorizzare l'input/output di un'istanza in esecuzione.
- **Esecuzione:** interfaccia per applicare un modello a un insieme di input e raccogliere l'output.

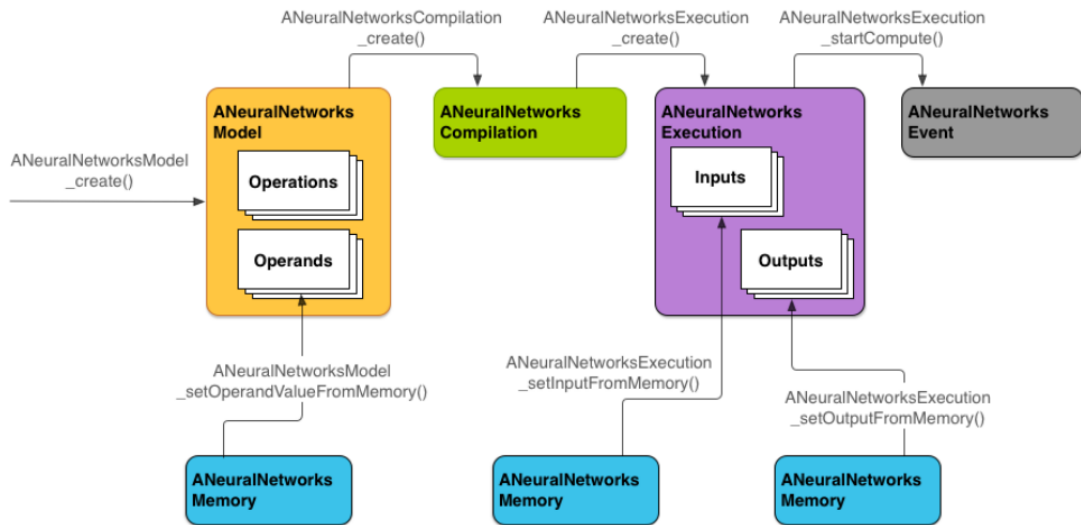


Figura 1.2: Flusso di programmazione per l'API Android Neural Networks

1.2.2 Modello: Operandi e Operazioni

Ogni modello è definito da:

- **Operandi:** oggetti di dati per la definizione del grafico, inclusi input/output, costanti e nodi intermedi. Ogni operando ha un tipo ed è identificato implicitamente per ordine. Esistono due tipi di operandi: scalari e tensori. Un valore scalare rappresenta un singolo valore mentre i tensori sono array n-dimensionali.
- **Operazioni:** specificano i calcoli da eseguire. Ogni operazione ha un tipo (somma, moltiplicazione, convezione), un elenco di indici degli operandi utilizzati per input/output.

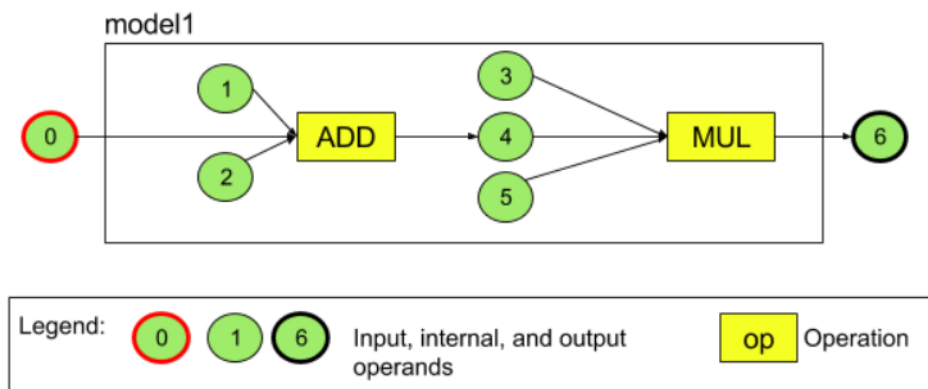


Figura 1.3: Operandi modello NNAPI

1.3 Vantaggi e Svantaggi dell’Inferenza On-Device

Vantaggi:

- **Latenza:** nessuna necessità di una connessione di rete e attesa della risposta.
- **Disponibilità:** Il funzionamento dipende solo dalle condizioni del dispositivo.
- **Velocità:** i nuovi acceleratori dedicati rendono molto efficiente l’inferenza.
- **Privacy:** I dati non vengono inviati ma rimangono in locale.
- **Costo:** Nessun bisogno di un server esterno.

Svantaggi:

- **Batteria:** l’uso delle reti neurali richiede molti calcoli, i quali potrebbero influenzare il consumo della batteria.
- **Spazio:** presenza di modelli di grandi dimensioni, che influenzano l’impatto sugli utenti.

L’NNAPI è usata da framework come TensorFlow Lite e ML Kit. In TensorFlow Lite, può essere utilizzata come delegato, ma le prestazioni non sono sempre migliori. Il comportamento dipende dal supporto del modello.

Capitolo 2

TensorFlow Lite

2.1 Cos'è e a cosa serve

TensorFlow Lite è un insieme di strumenti che consente il machine learning on-device, aiutando gli sviluppatori a eseguire i loro modelli su dispositivi mobili, embedded e IoT. È considerato una versione leggera di TensorFlow ed è progettato per operare su dispositivi con minima potenza di elaborazione e risorse limitate di calcolo e memoria. TensorFlow Lite esegue modelli addestrati su smartphone (Android e iOS), microcontrollori, dispositivi IoT e computer (Linux), in casistiche in cui TensorFlow standard ha delle limitazioni.

2.1.1 Workflow dello sviluppatore

Il processo di sviluppo con TensorFlow Lite include i seguenti passaggi principali:

1. Scegliere un modello.
2. Convertire il modello.
3. Eseguire l'inferenza con il modello.
4. Ottimizzare il modello.

Un modello TensorFlow è una struttura dati che contiene la logica e la conoscenza di una rete di machine learning addestrata. Per utilizzarlo con TensorFlow Lite, è necessario convertirlo nel relativo formato, poiché non è possibile creare o addestrare modelli direttamente in TensorFlow Lite. Il team di TensorFlow Lite fornisce modelli pre-addestrati per varie attività come classificazione delle immagini, rilevamento di oggetti, risposta intelligente, stima della posa e segmentazione, oltre a modelli disponibili su TensorFlow Hub, che potrebbero richiedere una conversione.



Figura 2.1: Modello utilizzato per l'app

Quando si desidera personalizzare un modello pre-addestrato per un'attività specifica, è possibile utilizzare il *Transfer Learning*, una tecnica che consente di riaddestrare un modello esistente con un numero ridotto di dati e in tempi più rapidi.

La libreria Model Maker facilita questo processo, permettendo di addestrare modelli personalizzati a partire da modelli già disponibili. I modelli creati e addestrati in TensorFlow, tramite API come Keras o Model Maker, devono essere successivamente convertiti in TensorFlow Lite per essere eseguiti su dispositivi mobili o embedded.

Questa conversione, effettuata tramite il TensorFlow Lite Converter (disponibile come API Python o da riga di comando, con la modalità Python raccomandata), riduce le dimensioni del modello e ne ottimizza l'esecuzione.

Sono supportati diversi formati in input, tra cui SavedModel, Frozen GraphDef, modelli Keras HDF5 e Concrete Functions.

Tuttavia, non tutti i modelli sono direttamente convertibili: se il modello contiene operazioni non supportate da TensorFlow Lite, è possibile utilizzare TensorFlow Select (che include una porzione del runtime di TensorFlow ma aumenta la dimensione del binario) oppure definire operatori personalizzati.

2.2 Esecuzione dell'inferenza

L'inferenza esegue i dati nel modello per ottenere previsioni. È necessario:

- il modello;
- un interprete;
- i dati di input.

L'interprete TensorFlow Lite supporta diversi linguaggi: Java, Swift, Objective-C, C++ e Python. I dispositivi dotati di una GPU possono usare i *delegati* (GPU Delegate, NNAPI, ecc.) per migliorare la velocità dell'inferenza.

2.2.1 Piattaforme supportate

- **Android e iOS:** supporto ufficiale con AAR e CocoaPods.
- **Linux:** utile per ML embedded, con interprete Python (tflite-runtime) e compilazione C++ per Raspberry Pi.
- **Microcontrollori:** TensorFlow Lite for Microcontrollers, pensato per dispositivi con memoria ridotta.

L'ottimizzazione dei modelli è fondamentale per ridurre le dimensioni e migliorare le prestazioni, soprattutto su dispositivi mobili o embedded. A tal fine, si utilizza il Model Optimization Toolkit, che include tecniche avanzate come la **quantizzazione**.

Questa tecnica riduce la precisione numerica dei pesi e delle attivazioni, ad esempio da float32 a int8, consentendo di diminuire la latenza e l'uso di memoria, mantenendo comunque un livello di accuratezza accettabile.

Si propone un'esempio di codice utilizzato per provare la quantizzazione, purtroppo però il risultato non ha soddisfatto le aspettative:

```
package com.example.prova_modello

import android.os.Bundle
import android.util.Log
import android.widget.TextView
import androidx.activity.ComponentActivity
import org.tensorflow.lite.Interpreter
import java.io.FileInputStream
import java.nio.ByteBuffer
import java.nio.ByteOrder
import java.nio.channels.FileChannel
import android.content.Context
import org.json.JSONObject
import java.nio.charset.StandardCharsets

class MainActivity : ComponentActivity() {
    private lateinit var gpt2Decoder: Gpt2Decoder

    override fun onCreate(savedInstanceState: Bundle?) {
```

```

super.onCreate(savedInstanceState)
setContentView(R.layout.activity_main)
val outputTextView: TextView = findViewById(R.id.outputTextView)

val interpreter = loadTFLiteModel("gpt2_model.tflite")

if (interpreter == null) {
    Log.e("TFLite", " Errore nel caricamento del modello!")
    return
}

val inputShape = interpreter.getInputTensor(0).shape()
val outputShape = interpreter.getOutputTensor(0).shape()

gpt2Decoder = Gpt2Decoder(this)

val prompt = "Give me the recipe of carbonara"
val tokens = prompt.split(" ").mapNotNull { word ->
    val tokenPiece = word
    gpt2Decoder.tokenToId["Ġ$tokenPiece"]
}

val inputBuffer = ByteBuffer.allocateDirect(4).
order(ByteOrder.nativeOrder())
inputBuffer.putInt(tokens[0])

val outputBufferSize = 4 * outputShape.sum()
val outputBuffer = ByteBuffer.allocateDirect(outputBufferSize).
order(ByteOrder.nativeOrder())
try {
    interpreter.run(inputBuffer, outputBuffer)
} catch (e: Exception) {
    Log.e("TFLite", " Errore durante l'inferenza: ${e.message}")
}

```

```

        return
    }

    val generatedText = decodeTokens(outputBuffer)

    outputTextView.text = generatedText
}

private fun loadTFLiteModel(fileName: String): Interpreter? {
    return try {
        val assetFileDescriptor = assets.openFd(fileName)
        val fileInputStream = FileInputStream(assetFileDescriptor.
            fileDescriptor)
        val fileChannel = fileInputStream.channel
        val modelBuffer = fileChannel.map(FileChannel.
            MapMode.READ_ONLY,
            assetFileDescriptor.startOffset,
            assetFileDescriptor.declaredLength)
        Interpreter(modelBuffer)
    } catch (e: Exception) {
        Log.e("TFLite", "Errore nel caricamento del modello:
            ${e.message}")
        null
    }
}

private fun decodeTokens(outputBuffer: ByteBuffer): String {
    outputBuffer.rewind()
    val outputArray = IntArray(outputBuffer.remaining() / 4)

    val rawTokens = mutableListOf<Int>()
    for (i in outputArray.indices) {
        outputArray[i] = outputBuffer.int and 0xFFFF
        rawTokens.add(outputArray[i])
    }
    return gpt2Decoder.decode(outputArray.toList())
}

```

}
}

Nella figura che segue è riportato l'output di una richiesta effettuata con il modello quantizzato. Il risultato è alquanto scadente e probabilmente è dovuto da un errore nella quantizzazione.

```
fru differentiated stress unableper fuelette  
overlyorsierenn trustingreetingsaking  
microironically Eur Cloneitto permanent  
Almost callsuras acupuncturecoveredfelt  
Returns stricken Hercoldlines Leafs  
sentiment appliance fiber Ros  
inherently Bra negatively Petroleum  
endings fascist goalt Portugal  
scenarios Rihhello intelligence  
accomplishments prohibitionsijah  
entails displaced Giryoutu SequenceAs  
630losadigyinatingeks riskedSweet32later  
ColtsekiArmorhibitedaminationvr wear  
inex AlexandriaFFER Ambro rearEEP  
astroph Albert Harding agric tuberculosis  
circuit Doyle mastering Slaughter painfully  
specimen Zhuixon EVsOM Yorker 425  
fors=$ passages Homs smile Justice  
AFachus Crimea Trin Reducediber  
proceeding Humanity allegiance imprint  
Melaniaaklākī Marinersstaff laundustrial  
thou Bolivia reconciliationhealthy  
Barack canopy playersStreet  
hiddenateful Grass USSIRC roster  
liberating actu isoDetect 1830orescent  
vanish Lumin masteringresources  
Blendlethalastonprototype  
diligence-----  
AlvarezãH© Categories collaborators  
long worsened Alpine Payments bonded  
timeframe reptiles468Lasburning  
chaired Hastepriison Radiationlocalhost  
Esther SkipINO condensed Wit QUEST  
stocked 71 cheap HYsov intentionally
```

Figura 2.2: Risultati quantizzazione

Per addestrare un modello TensorFlow Lite su un dataset personalizzato, è disponibile una libreria che semplifica notevolmente il processo. Questa libreria supporta diverse attività di machine learning, tra cui la classificazione di immagini, il rilevamento di oggetti, la classificazione di testo e audio, la risposta a domande e i sistemi di suggerimento. Inoltre, esistono API progettate che offrono un'interfaccia semplice e ben definita, facilitando l'integrazione dei modelli di machine learning all'interno delle applicazioni. [2]

2.3 Differenze principali da TensorFlow

- **Ambiente di esecuzione:** TensorFlow è pensato per sistemi come server/cloud mentre TensorFlow Lite per dispositivi mobili/IoT.
- **Addestramento:** TensorFlow riesce ad addestrare il modello mentre TensorFlow Lite lavora solo con modelli già addestrati.
- **Latenza:** TensorFlow Lite ha una latenza inferiore rispetto a TensorFlow, il che lo rende ideale per applicazioni real-time.

- **Formato del modello:** TensorFlow Lite usa `.tflite` (FlatBuffer), non convertibile in TensorFlow.
- **Velocità on-device:** TensorFlow Lite è più veloce su dispositivi con risorse limitate, visto che è più leggero rispetto a TensorFlow.

Capitolo 3

LiteRT

3.1 Cos'è LiteRT

LiteRT, abbreviazione di Lite Runtime e precedentemente noto come TensorFlow Lite, è il runtime ad alte prestazioni sviluppato da Google per l'intelligenza artificiale on-device.

Fa parte dell'iniziativa Google AI Edge ed è progettato per eseguire modelli di machine learning su dispositivi con risorse computazionali limitate, come smartphone, microcontrollori, dispositivi IoT e sistemi embedded basati su Linux.

3.1.1 Funzionalità principali

- **Ottimizzazione per il machine learning on-device:** LiteRT risolve i principali problemi dei modelli on-device come latenza, privacy dimensioni connettività e consumo energetico.
- **Supporto multiplatforma:** compatibilità con Android e Ios, Linux embedded e microcontrollori.
- **Opzioni di modelli multi framework:** fornisce strumenti per la conversione di modelli da Tensorflow, PyTorch e JAX al formato FlatBuffers utilizzato da LiteRT. Ha anche accesso a strumenti per la gestione della quantizzazione del modello e dei metadati.
- **Supporto di diversi linguaggi:** include SDK per Java/Kotlin, Swift, Objective-C, C++ e Python.
- **Elevate prestazioni:** accelerazione hardware tramite delegati come GPU e Core ML per iOS.

3.1.2 Flusso di lavoro di sviluppo

Il flusso di lavoro di sviluppo con LiteRT prevede tre passaggi principali:

- **Identificare la soluzione più adatta al problema:** LiteRT propone due diverse personalizzazioni per la risoluzione del problema. Le principali soluzioni sono usare MediaPipe tasks che fornisce soluzioni pronte all'uso per attività di machine learning oppure LiteRT che richiede di convertire il modello e di andare ad eseguirlo sul dispositivo.
- **Scelta di un modello:** un modello LiteRT è in un formato portatile chiamato FlatBuffers che ha come estensione `.tflite`. Esistono due modi per utilizzare un modello su LiteRT: utilizzare un modello già esistente nel formato `.tflite` oppure convertire un modello in un modello adatto a LiteRT seguendo le varie istruzioni della documentazione ufficiale. I principali siti su cui trovare i vari modelli sono: Kaggle Models, Hugging Face e torchvision.
- **Integrazione all'interno dell'app:** LiteRT contiene API per Python, Java e Kotlin, Swift e C++ che ti permettono di eseguire inferenze completamente sui tuoi dispositivi.

I modelli possono includere metadati opzionali che offrono descrizioni leggibili sia dall'uomo che dalla macchina, utili per automatizzare le fasi di pre e post-elaborazione dei dati.

3.1.3 Conversione dei modelli

Per utilizzare i modelli addestrati in TensorFlow (e in altri framework) questi devono essere prima convertiti nel formato `.tflite` usando il convertitore LiteRT.

I principali modi con cui puoi convertire un modello sono:

- **API Python:** ti consente di integrare la conversione della pipeline di sviluppo, applicare ottimizzazioni, aggiungere metadati e altre attività che semplificano la conversione.
- **Riga di comando:** supporta solo la conversione del modello di base. Il comando è `tf.lite.TFLiteConverter`

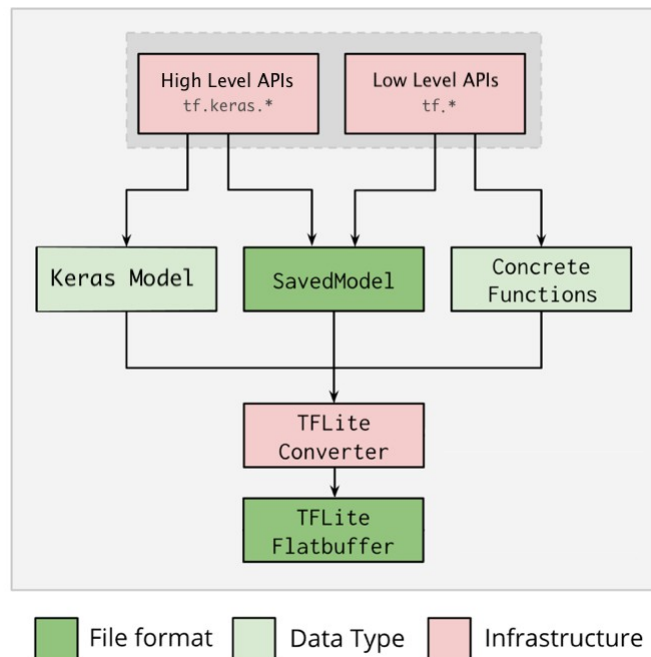


Figura 3.1: Scema conversione modello

Inoltre esiste un'API sperimentale per la conversione da modelli JAX, che precedentemente non erano supportati.

Durante la conversione, è fondamentale valutare la compatibilità degli operatori usati nel modello, poiché la libreria di operatori di LiteRT è un sottoinsieme di quella di TensorFlow Core e potrebbero esserci degli operatori non presenti in TensorFlow Core. Il convertitore LiteRT è progettato per analizzare il modello strutturare e applicare ottimizzazioni in modo da renderla compatibile operatori supportati direttamente.

Nell'insieme degli operatori esistono tre tipi che sono supportati dal processo di conversione:

1. Modelli con solo operatori integrati in LiteRT (i più consigliati).
2. Modelli con operatori integrati + operatori selezionati da TensorFlow Core;
3. Modelli con operatori integrati + operatori TensorFlow Core e/o operatori personalizzati.

L'opzione più stabile è utilizzare solo operatori integrati, evitando flag aggiuntivi e semplificando l'ottimizzazione. L'inclusione di operatori TensorFlow richiede il caricamento del runtime TensorFlow, aumentando la dimensione del binario.

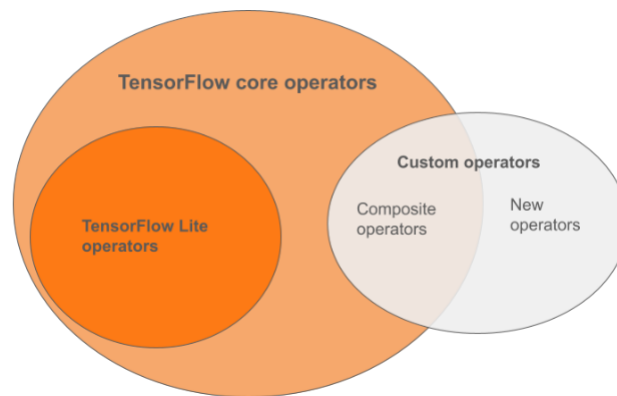


Figura 3.2: Insiemi degli operatori

3.1.4 Esecuzione dell'inferenza

Un modello LiteRT in esecuzione all'interno di un'app android acquisisce i dati, li elabora e genera una previsione in base alla logica del modello. Per l'esecuzione di un modello LiteRT è necessario un ambiente di runtime speciale e i dati trasmessi al modello devono essere in un formato specifico, chiamato tensore. Quando un modello elabora i dati, ovvero esegue un'inferenza.

L'inferenza con LiteRT richiede un modello `.tflite`, un interprete e i dati di input. L'interprete carica il modello, esegue il grafo e restituisce i risultati utilizzando un grafo statico ed un allocatore di memoria ottimizzato per ottenere la minima latenza possibile. Le API dell'interprete sono disponibili nei principali linguaggi di programmazione. Su Android si utilizzano Java, Kotlin o C++ ed il modello può essere caricato da file o da memoria (rispettivamente `MappedByteBuffer` in Java/Kotlin o `FlatBufferModel` in C++).

I tipi di dato supportati includono `float`, `int`, `long`, `byte` e `String`. Se il modello ha una forma dinamica di input, è necessario ridimensionare la forma prima dell'inferenza. L'accelerazione hardware può essere abilitata attraverso l'interprete.

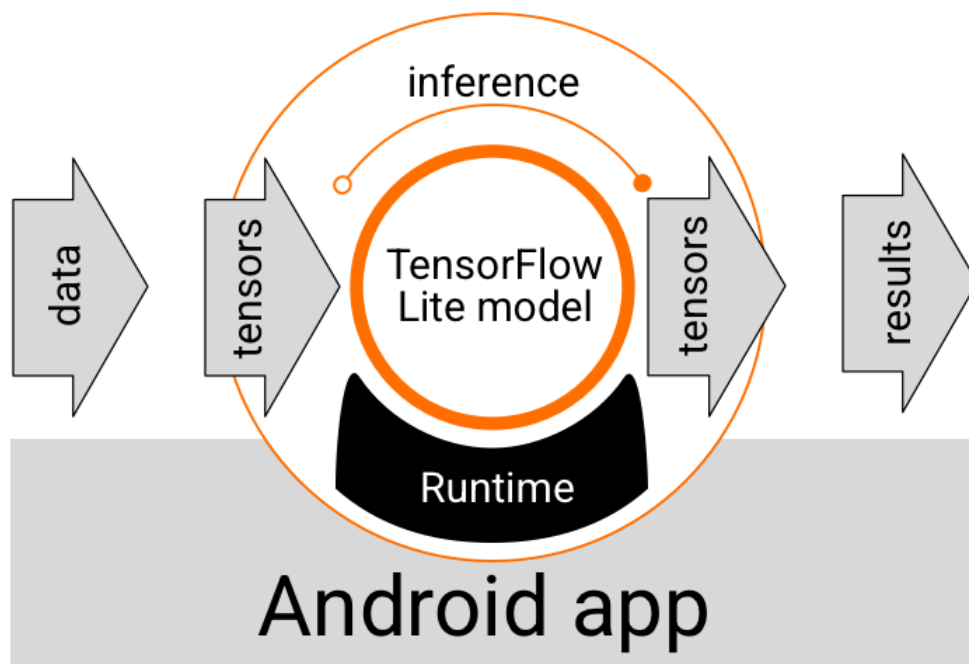


Figura 3.3: Esempio inferenza in LiteRT

3.1.5 Migrazione da TensorFlow Lite

Le API di LiteRT mantengono la stessa nomenclatura delle API di TF Lite, quindi la migrazione richiede solo modifiche minime. In particolare, per Android è sufficiente sostituire la dipendenza `org.tensorflow:tensorflow-lite` con `com.google.ai.edge.litert` nel file `build.gradle`.

3.2 Quantizzazione in LiteRT

La quantizzazione è una strategia fondamentale di ottimizzazione in LiteRT. Essa converte i numeri in virgola mobile a 32 bit (come i pesi e gli output di attivazione) in rappresentazioni a precisione ridotta, in genere numeri interi a 8 bit. L'obiettivo principale è ridurre le dimensioni del modello e aumentare la velocità di inferenza, una caratteristica particolarmente utile per dispositivi a bassa potenza come i microcontrollori. Inoltre, questo formato è necessario per acceleratori hardware come l'Edge TPU, che supportano solo operazioni basate su numeri interi.

LiteRT supporta diverse strategie di quantizzazione che possono essere applicate ai modelli dopo il processo di addestramento.

Quantizzazione dell'intervallo dinamico

Questa tecnica converte i pesi del modello in valori interi a 8 bit durante la conversione da un grafico TensorFlow al formato FlatBuffer. Le attivazioni rimangono in virgola mobile, ma vengono quantizzate e dequantizzate dinamicamente durante l'inferenza, permettendo l'utilizzo di kernel quantizzati più efficienti. Si tratta di una strategia semplice che può ridurre le dimensioni del modello fino a quattro volte, mantenendo l'accuratezza. Per esempio GPT2-medio con la quantizzazione presentata nel codice sopra è passato 500MB a 160MB.

Quantizzazione float16

In questo caso, i pesi vengono convertiti in virgola mobile a 16 bit (float16), riducendo le dimensioni del modello di circa la metà. Questa tecnica è particolarmente vantaggiosa su hardware in grado di eseguire operazioni nativamente con float16, come le GPU. I modelli quantizzati in float16 sono eseguibili anche su CPU, dove i pesi vengono convertiti a float32 all'inizio dell'inferenza. L'impatto su accuratezza e latenza è in genere minimo, con un buon compromesso tra dimensioni e prestazioni.

Quantizzazione intera completa (Full Integer)

Questa tecnica converte sia pesi che attivazioni in numeri interi a 8 bit. Richiede la fornitura di un RepresentativeDataset per permettere al convertitore di stimare i range dinamici. Anche se l'intero grafo può essere quantizzato, per impostazione predefinita i tensori di input e output rimangono in virgola mobile. Questo rende il modello incompatibile con hardware che supporta solo numeri interi, a meno che non si specifichi esplicitamente il tipo di input/output come interi.

Quantizzazione solo per numeri interi

Per ottenere un modello interamente quantizzato in interi (inclusi input e output), è necessario impostare:

- `converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]`
- `converter.inference_input_type = tf.uint8`
- `converter.inference_output_type = tf.uint8`

Questa configurazione garantisce compatibilità con acceleratori come l'Edge TPU. In test eseguiti su un modello MNIST, i risultati in termini di accuratezza sono risultati simili a quelli della versione float.

3.2.1 Specifiche tecniche della quantizzazione a 8 bit

LiteRT segue uno schema preciso per la quantizzazione a 8 bit, fondamentale per garantire interoperabilità e prestazioni ottimali su hardware dedicato.

I pesi sono rappresentati da valori `int8` nel range $[-127, 127]$, con punto zero fissato a 0 (quantizzazione simmetrica). Le attivazioni e gli input, invece, sono nel range $[-128, 127]$ con punto zero variabile (quantizzazione asimmetrica). Per i pesi, è possibile usare granularità per asse (es. per canale) o per tensore; per le attivazioni, solo per tensore.

LiteRT privilegia l'uso di `int8`, per sfruttare la semplicità della quantizzazione simmetrica e le ottimizzazioni hardware esistenti. L'approssimazione del valore reale avviene secondo la formula:

$$\text{real_value} = (\text{int8_value} - \text{zero_point}) \times \text{scale}$$

3.2.2 Operatori quantizzati

La specifica dei requisiti per operatori quantizzati definisce input, pesi, bias e output. Ad esempio, l'operatore `CONV_2D` prevede:

- Input (attivazione): `int8` per tensore
- Pesi: `int8` per asse (`dim=0`), `zero_point = 0`
- Bias: `int32` per asse, con scala derivata
- Output: `int8` per tensore

Queste specifiche consentono una quantizzazione efficace con controllo dettagliato sulla granularità e sui parametri.

3.2.3 Quantization Debugger

Quando la qualità del modello quantizzato è inferiore alle aspettative, LiteRT mette a disposizione il **Quantization Debugger**, uno strumento diagnostico avanzato che consente di:

- Analizzare le metriche di qualità della quantizzazione per ciascun tensore.
- Confrontare direttamente gli output float e quantizzati.
- Identificare layer problematici usando metriche come RMSE normalizzato.

Il debugger si integra con l'API `TFLiteConverter` e utilizza lo stesso `RepresentativeDataset` impiegato per la quantizzazione intera. I risultati possono essere esportati in formato CSV o analizzati via codice (es. con `pandas`). Una modalità avanzata consente anche la verifica dell'intero modello (anziché layer per layer), utile per osservare come l'errore si propaga lungo il grafo computazionale.

3.3 Metadati LiteRT

I metadati LiteRT forniscono uno standard per la descrizione dei modelli, costituendo una fonte di conoscenza sia per l'utente umano che per gli strumenti automatici. Sono definiti utilizzando il formato FlatBuffer (`metadata_schema.fbs`) e archiviati nel campo `metadata` del modello TFLite con il nome `TFLITE_METADATA`. Nei modelli pubblicati su Kaggle e HuggingFace, tutti quelli relativi a immagini sono compilati con metadati.

Lo schema dei metadati comprende tre componenti principali: le informazioni generali sul modello, incluse la descrizione e la licenza; i dettagli sugli input, come il tipo e la pre-elaborazione (ad esempio la normalizzazione); e infine le informazioni sugli output, tra cui le tecniche di post-elaborazione come la mappatura alle etichette.

Model

Name	aiy/vision/classifier/food_V1
Description	Fine grained classification model for recognizing food dishes.
Version	1
Author	Google
License	Apache-2.0

Tensors

Inputs

Name	Type	Description	Shape	Min / Max
image	Image <uint8>	Input image to be classified. The expected image is 224 x 224 with 3 channels per pixel, with each value in [0.0,255.0].	[1, 192, 192, 3]	[0] / [255]

Outputs

Name	Type	Description	Shape	Min / Max
probability	Feature <uint8>	Probabilities of the outputs classes.	[1, 2024]	[0] / [1]

Figura 3.4: Metadati del modello utilizzato per la classificazione del cibo

Attualmente LiteRT supporta solo un singolo sottografo. Di conseguenza, strumenti come Android Studio ML Binding e i generatori di codice si affidano a `ModelMetadata.name` e `ModelMetadata.description` invece che a `SubGraphMetadata`.

I metadati sono progettati per tipologie di input/output piuttosto che per modelli specifici. Sono supportati input numerici (interi senza segno e float32), immagini RGB o in scala di grigi, e riquadri di delimitazione con vari schemi di numerazione.

I modelli LiteRT possono avere file associati (ad esempio, file di etichette o vocabolari), essenziali per il corretto funzionamento. Questi file possono essere integrati nei metadati e raggruppati con il modello in un file con estensione `.tflite`, compatibile con TFLite e strutturato come un file ZIP. Ciò consente al codice generato di includere automaticamente le procedure di pre e post-elaborazione.

3.4 Aggiunta di Metadati

È necessario un ambiente Python con TensorFlow e il pacchetto `tflite-support`. Per usare l'API Metadata Writer, è richiesta la versione `tflite-support-nightly`.

Esistono due principali modalità per aggiungere metadati: l'API Flatbuffers (l'immagine sopra) e l'API Metadata Writer. Quest'ultima è più semplice ed è adatta per attività comuni supportate dalla Task Library, tra cui classificatori di immagini, rilevatori di oggetti, classificatori audio e linguistici. Dopo la creazione dei FlatBuffers, si utilizzano strumenti come `MetadataPopulator` per scrivere i metadati nel file `.tflite`.

3.5 Visualizzazione e Lettura dei Metadati

La visualizzazione può avvenire tramite strumenti come Netron o attraverso l'esportazione dei metadati in JSON con `MetadataDisplayer`. Android Studio consente la visualizzazione grazie al ML Binding.

La lettura dei metadati può essere eseguita tramite la libreria `Metadata Extractor`, disponibile per Java e C++. In Java, si utilizza il pacchetto `org.tensorflow:tensorflow-lite-metadata` da MavenCentral. Un oggetto `MetadataExtractor` inizializzato con un `ByteBuffer` del modello consente l'accesso ai metadati di input/output, inclusi forma e parametri di quantizzazione.

```
implementation 'org.tensorflow:tensorflow-lite:2.12.0'  
implementation 'org.tensorflow:tensorflow-lite-support:0.4.2'
```

Figura 3.5: Esempio implementazione codice di supporto in Kotlin

Il controllo versione dello schema è garantito da un numero semantico (MAJOR.MINOR.PATCH) e un identificatore di 4 caratteri nel file Flatbuffers. Un parser obsoleto potrebbe non riuscire a leggere i campi aggiunti successivamente, anche se l'identificatore combacia. La versione minima necessaria del parser è definita automaticamente da MetadataPopulator. [3]
[4]

Capitolo 4

MediaPipe

4.1 MediaPipe Solutions

MediaPipe Solutions è una suite di librerie e strumenti progettata per applicare rapidamente tecniche di intelligenza artificiale (IA) e machine learning (ML) alle applicazioni. Fa parte del progetto open source MediaPipe. Con MediaPipe Solutions è possibile collegare le soluzioni alle applicazioni, personalizzarle in base alle proprie esigenze e utilizzarle su più piattaforme di sviluppo.

La suite include diverse componenti chiave. La prima è **MediaPipe Tasks**, che fornisce le API e le librerie multiplatforma fondamentali per il deployment delle soluzioni. Si tratta di API ben definite che permettono di eseguire l'inferenza ML in modo semplice, anche con poche righe di codice. MediaPipe Tasks non si limita all'inferenza di un singolo modello, ma offre pipeline ML ottimizzate per prestazioni elevate e accelerazione su CPU, GPU e TPU. È progettata per un uso in tempo reale su dispositivo. Le API MediaPipe Tasks sono disponibili per Android, Web/JavaScript e Python.

```
.setPreferredBackend(LlmInference.Backend.CPU)
```

Figura 4.1: Riga di codice che specifica l'utilizzo di CPU/GPU

La seconda componente è rappresentata dai **Modelli MediaPipe**, che comprendono modelli preaddestrati pronti all'uso e integrabili con ogni soluzione.

Un'altra componente fondamentale è **MediaPipe Model Maker**, uno strumento per personalizzare modelli ML esistenti utilizzando dati specifici per una determinata applicazione. Questo approccio costituisce un'alternativa più rapida alla creazione e all'addestramento di un modello ex novo. MediaPipe Model Maker sfrutta il *Transfer Learning*, una tecnica che

riaddestra i modelli esistenti rimuovendo gli ultimi livelli e ricostruendoli in base ai nuovi dati forniti. Questo processo consente di riutilizzare gran parte della logica già presente nel modello originario, riducendo notevolmente la quantità di dati e tempo necessari per l'addestramento. In genere, sono richiesti circa 100 campioni per classe, anche se talvolta ne bastano meno. Il risultato è spesso un modello più leggero, preciso e mirato. Model Maker supporta vari tipi di modelli, tra cui classificatori di immagini, riconoscitori di gesti, modelli per il testo e modelli audio.

Tuttavia, è importante notare che i modelli personalizzati possono riconoscere solo le classi presenti nei nuovi dati e non possono cambiare il compito per cui erano stati originariamente progettati. Ad esempio, non è possibile trasformare un classificatore d'immagini in un rilevatore di oggetti.

L'intera suite è progettata per supportare l'inferenza ML on-device. A tal fine è stato introdotto un nuovo set di API, denominato **LiteRT Next**, che semplifica l'uso dell'accelerazione hardware su dispositivi locali.

Le attività ML supportate da MediaPipe Tasks coprono diversi domini applicativi come la visione artificiale, il linguaggio naturale e l'audio. Le soluzioni disponibili comprendono l'inferenza con *Large Language Models* (LLM):

- il rilevamento di oggetti
- la classificazione e la segmentazione delle immagini
- il riconoscimento dei gesti e dei punti di riferimento della mano
- l'incorporamento delle immagini
- il rilevamento e la stilizzazione dei volti

È importante sottolineare che l'attuale versione di MediaPipe Solutions è una **Preview**. Come tale, può presentare un supporto limitato, introdurre modifiche non retrocompatibili e subire variazioni nella disponibilità senza preavviso.

Questa nuova architettura di MediaPipe Solutions rappresenta una convergenza di strumenti esistenti, unificando le soluzioni precedenti di MediaPipe, la Task Library di TensorFlow Lite e il Model Maker di TensorFlow Lite in un'unica interfaccia coerente e moderna. [5]

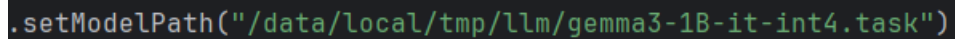
4.2 MediaPipe API

MediaPipe, insieme a LiteRT, fa parte delle Soluzioni di Google AI Edge. LiteRT Next è un nuovo insieme di API che migliora e semplifica l'accelerazione hardware on-device. Nel caso in cui si abbia la necessità di inserire manualmente il modello all'interno del dispositivo si può utilizzare il seguente codice da un terminale (sia Bash che AndroidStudio) :

```
adb shell rm -r /data/local/tmp/llm/  
adb shell mkdir -p /data/local/tmp/llm/  
adb push output_path /data/local/tmp/llm/model_version.task
```

Le attività MediaPipe sono utilizzate per creare applicazioni Android e richiedono una configurazione specifica dell'ambiente di sviluppo, tra cui Android Studio (versione consigliata 2021.1.1 Bumblebee o superiore), SDK Android versione 24 o successiva, e un dispositivo Android fisico con la versione minima dell'SDK. È necessario attivare le Opzioni sviluppatore e il debug USB su un dispositivo fisico.

MediaPipe Tasks fornisce tre librerie predefinite: visione artificiale, testo e audio. I file modello con estensione `.tflite` devono trovarsi nella directory degli asset Android dell'applicazione oppure va specificato il *path* da codice.

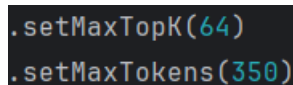


```
.setModelPath("/data/local/tmp/llm/gemma3-1B-it-int4.task")
```

Figura 4.2: Esempio riga di codice che indica path per il modello

Le attività di IA generativa comprendono la generazione di immagini (libreria `tasks-vision-image-generator`) e l'inferenza LLM (libreria `tasks-genai`), che consente l'esecuzione di Large Language Models completamente on-device per Android. Queste API sono ottimizzate per dispositivi di fascia alta e non funzionano in modo affidabile sugli emulatori. **googleai2024**

È possibile configurare le API MediaPipe Tasks tramite `BaseOptions`, con opzioni come `modelPath`, `maxTokens`, `topK`, `temperature`, `randomSeed` e l'abilitazione dell'accelerazione hardware.



```
.setMaxTopK(64)  
.setMaxTokens(350)
```

Figura 4.3: Esempio di codice utilizzato

4.3 OutputHandler

Si tratta di una classe pubblica fornita dalle API **MediaPipe** di Google AI Edge. Rappresenta la classe base per la gestione degli output prodotti dai grafici di attività e fa parte dell'infrastruttura *LiteRT Next*.

Questa classe consente la trasformazione dei pacchetti di output in oggetti risultato e la definizione di callback personalizzati per la gestione degli eventi e delle eccezioni.

La classe contiene diverse interfacce nidificate, ognuna pensata per gestire specifici tipi di output o comportamenti legati all'elaborazione dei task.

- **OutputPacketConverter<OutputT, InputT>** gestisce la conversione di ERROR(/Packet) di output del grafico MediaPipe in oggetto risultato attività e oggetto di input dell'attività.
- **ProgressListener<OutputT>** è il listener dei risultati delle attività MediaPipe che riceve aggiornamenti parziali delle attività fino a quando non viene richiamato con "done" impostato su true.
- **ResultListener<OutputT, InputT>** è utilizzata dal listener dei risultati delle attività MediaPipe, in grado di recuperare sia gli oggetti dei risultati delle attività sia i dati di input corrispondenti.
- **ValueListener<OutputT>** riceve solo il valore di output di un'attività.

4.4 ProgressListener<String>

La gestione dei dati parziali è particolarmente utile non solo per monitorare l'avanzamento delle operazioni, ma anche per fornire all'utente un riscontro continuo sullo svolgimento dell'attività in corso, migliorando così la percezione di reattività e trasparenza del sistema. In particolar modo nella gestione delle risposte asincrone del sistema il ProgressListener risulta fondamentale in questi aspetti, perchè oltre alla risposta *token* per *token*, si può implementare un reparto grafico che mostra il progresso della risposta attesa.

```

llmInference?.generateResponseAsync(
    prompt,
    ProgressListener<String>{ partialResult, done ->

        currentTokens++
        val progressPercent = (currentTokens * 100) / maxTokens
        runOnUiThread {
            resultBuilder.append(partialResult)

            progressBar.progress = progressPercent

            if (done) {
                progressBar.progress = 100
                val intent = Intent( packageContext: this@RecipeLoadingActivity, RecipeResultActivity::class.java)
                if(imageUriString != null)
                    intent.putExtra( name: "imageURI",imageUriString)
                intent.putExtra( name: "inference_result", resultBuilder.toString())
                intent.putExtra(FOOD_NAME_KEY, foodName)
                intent.putExtra(IS_RANDOM_RECIPE_KEY, isRandomRecipe)
                startActivity(intent)
                finish()
            }
        }
    })
}

```

Figura 4.4: Esempio codice per gestione risposta asincrona

4.5 Gestione risorse

La gestione dei thread è anch'essa importante per ottimizzare le prestazioni, specialmente in applicazioni mobile o real-time in cui l'utente necessita di compiere operazioni anche durante l'esecuzione di un'inferenza o di una richiesta. In particolare modelli caricati in locale sul dispositivo richiedono tempo prima di essere utilizzati ad ogni avvio di applicazione. L'idea è quindi quella di creare una sezione di codice dedicata per gestire tale caricamento, così da non bloccare le azioni dell'utente nel mentre e velocizzare le successive richieste che verranno effettuate al modello.

```

lifecycleScope.launch {
    llmInference = withContext(Dispatchers.IO){
        loadLlmModel()
    }
    if (llmInference != null) {
        startInference(foodName, imageUriString, isRandomRecipe, languageCode)
    } else {
        AlertDialog.Builder( context: this@RecipeLoadingActivity)
            .setTitle("Error")
            .setMessage("Failed to load the model.")
            .setPositiveButton( text: "OK", listener: null)
            .show()
    }
}

```

Figura 4.5: Esempio codice per caricare il modello

Capitolo 5

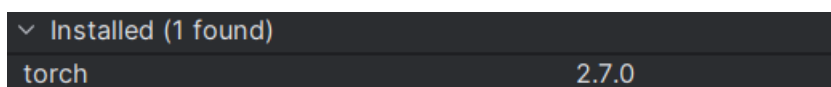
ExecuTorch

5.1 Introduzione

ExecuTorch è una piattaforma progettata per supportare l'inferenza su dispositivi mobile ed edge, consentendo l'esecuzione di modelli PyTorch in ambienti a risorse limitate. Si tratta di una soluzione end-to-end che copre l'intero ciclo di vita del modello, dalla conversione fino all'esecuzione su dispositivi embedded, smartphone o microcontrollori. La flessibilità della piattaforma consente l'ottimizzazione di modelli per numerosi backend hardware, inclusi CPU, GPU, NPU e DSP.

5.2 Flusso Operativo

L'utilizzo di ExecuTorch prevede una pipeline ben definita. Il primo passo consiste nell'installazione del pacchetto Python e delle librerie di runtime necessarie. Successivamente, il modello PyTorch viene esportato per una configurazione hardware specifica. Tale modello viene poi eseguito tramite le API di runtime, prima sulla piattaforma di sviluppo per test e validazione, e infine distribuito sulla piattaforma target. Questo approccio garantisce che il comportamento del modello sia coerente tra fase di sviluppo e produzione.



▼ Installed (1 found)	
torch	2.7.0

Figura 5.1: Pacchetto utilizzato

5.3 Esportazione e Trasformazione del Modello

Una delle caratteristiche principali di ExecuTorch è la possibilità di esportare modelli PyTorch nel formato `.pte`, ottimizzato per l'esecuzione su specifici backend hardware. Questo processo, definito di "abbassamento", è fondamentale per garantire l'efficienza computazionale su dispositivi a bassa potenza.

La procedura inizia con la selezione del backend target, seguita dalla preparazione del modello PyTorch. Quest'ultimo deve essere configurato in modalità `.eval()` e accompagnato da input di esempio con specifiche di forma. L'esportazione avviene tramite la funzione `torch.export.export`, mentre l'ottimizzazione per il backend target è affidata alla funzione `to_edge_transform_and_lower`. Il risultato finale, ottenuto con `to_executorch`, è un file `.pte` serializzato.

Il sistema è in grado di individuare automaticamente le partizioni del modello compatibili con il backend selezionato. Le parti non supportate vengono eseguite tramite un fallback portabile su CPU. Inoltre, la possibilità di specificare più partizionatori permette di gestire modelli complessi con strategie flessibili.

Gestione delle Forme Dinamiche e Quantizzazione

ExecuTorch supporta forme dinamiche, consentendo di definire dimensioni di input variabili, con l'imposizione di limiti per evitare sprechi di memoria. Durante la fase di esportazione, è possibile specificare tali limiti tramite il parametro `dynamic_shapes`.

Un altro aspetto rilevante è la quantizzazione, generalmente applicata in questa fase. Essa consente di ridurre le dimensioni del modello e accelerare l'inferenza, utilizzando precisioni numeriche inferiori. La quantizzazione, tuttavia, è sempre specifica del backend selezionato.

5.4 Backend Hardware Supportati

ExecuTorch è compatibile con una vasta gamma di backend hardware, ciascuno specializzato per differenti dispositivi. Tra i principali troviamo:

- **XNNPACK**, ottimizzato per CPU mobile (sia Arm che x86);
- **Core ML** e **MPS** (Metal Performance Shaders), entrambi specifici per dispositivi iOS;
- **Vulkan**, destinato all'uso su GPU Android;
- **Qualcomm AI Engine** e **MediaTek NeuroPilot**, due soluzioni per l'inferenza accelerata su NPU Android;

- **ARM Ethos-U** e **Cadence Xtensa**, rispettivamente per ambienti NPU e DSP.

Il supporto per backend multipli consente un’elevata portabilità e ottimizzazione mirata su differenti architetture hardware.

5.5 Esecuzione su Dispositivo

ExecuTorch offre API di runtime per Android (Java/Kotlin), iOS (Objective-C e C++) e C++.

Per Android, l’integrazione avviene tramite il pacchetto `executorch.aar`, disponibile su Maven Central. Questo contiene binding Java/Kotlin, le librerie core, il backend XNNPACK, kernel portabili e ottimizzati, oltre a librerie custom per modelli come LLaMa. È supportata la costruzione da sorgente tramite lo script `scripts/build_android_library.sh`, con compatibilità garantita per ABI `arm64-v8a` e `x86_64`.

5.6 Costruzione da Sorgente

ExecuTorch utilizza CMake come sistema di build principale e supporta Linux, macOS e Windows tramite WSL. La compilazione richiede Python 3.10–3.12 e un compilatore C++17 compatibile.

Dopo aver clonato il repository, è possibile installare le dipendenze con lo script `./install_executorch.sh`, specificando i backend desiderati tramite l’opzione `--pybind`. Per aggiornare il codice, è raccomandato l’uso dell’opzione `--clean`.

La compilazione viene effettuata in una directory dedicata e genera librerie come `libexecutorch.a`, `libportable_kernels.a` e `libportable_kernels_bindings.a`. Inoltre, viene costruito lo strumento `executor_runner`, utile per testare file `.pte`.

5.7 Integrazione del Runtime e Ottimizzazioni

Il runtime è altamente configurabile. Il logging, ad esempio, può essere abilitato o disabilitato a tempo di build usando opzioni CMake. ExecuTorch fornisce un Platform Abstraction Layer (PAL), che permette l’integrazione con le funzionalità di sistema host per logging, gestione errori e timing.

La piattaforma offre inoltre due tipologie di librerie di kernel: portabili (compatibili con tutti i backend) e ottimizzati (mirati per l’hardware specifico). Quando si compila da sorgente, è possibile scegliere quali includere, abilitando anche una *Selective Build* per ridurre le dimensioni del binario includendo solo gli operatori effettivamente utilizzati dal modello.[6]

Bibliografia

- [1] Android Developers, *Neural Networks API*, <https://developer.android.com/ndk/guides/neuralnetworks>, Accessed: 2025-06-15, 2025.
- [2] Google AI, *TensorFlow Lite Overview*, <https://ai.google.dev/edge/tflite/>, Accessed: 2025-05-23, 2024.
- [3] Google AI, *LiteRT Overview*, <https://ai.google.dev/edge/litert>, Accessed: 2025-06-15, 2024.
- [4] Android Developers, *LiteRT on Android*, <https://developer.android.com/ai/custom>, Accessed: 2025-05-23, 2024.
- [5] G. AI. «Panoramica del pacchetto `com.google.mediapipe.tasks.core` — MediaPipe Java API.» (2025), indirizzo: <https://ai.google.dev/edge/api/mediapipe/java/com/google/mediapipe/tasks/core/package-summary?hl=it>.
- [6] Pytorch, *ExecuTorch*, <https://docs.pytorch.org/executorch-overview>, Accessed: 2025-06-15, 2025.